

第2章 Python序列

主要内容：

一、元组

二、字典

2.0 序列(sequence)

- **序列**：是一系列值（对象），它们通常是相关的，以某种规则进行排列。
- 序列中每个组成部分称为“**元素**” (element)。
- 序列是程序设计中经常用到的数据存储方式，几乎每一种程序设计语言都提供了表格数据结构，如C和Basic中的一维、多维数组等。
- Python提供的序列类型在所有程序设计语言中是最丰富，最灵活，也是功能最强大的。

2.0 序列

- Python中常用的序列结构有列表、元组、字典*、字符串、集合*以及range等等。

	可变	不可变
有序	列表	元组**、字符串
无序	字典、集合	

* 在有些表述中，字典和集合不属于序列。

** 元组的元素（所引用对象）的值可变。

有序序列（列表、元组、字符串）支持双向索引，第一个元素下标为0，第二个元素下标为1，以此类推；最后一个元素下标为-1，倒数第二个元素下标为-2，以此类推。

2.0 序列

```
a_list = ['a', 'b', 'mpilgrim', 'z',  
          'example']
```

```
a_tuple = (element1, element2, ... ,  
           elementn)
```

```
a_dict = {'数学': 95, '英语': 92, '语文':  
          84}
```

```
a_set = {element1, element2, ..., elementn}
```

列表

```
suits =  
["♠", "♥", "♣", "♦"]
```

索引（下标）



0 1 2 3 ... n-1 ← 索引（下标）



$-n$ $-(n-1)$ $-(n-2)$... -2 -1 ← 索引（下标）

列表方法

list slicing

方法	说明
<code>list.append(x)</code>	将元素x添加至列表尾部
<code>list.extend(L)</code>	将列表L中所有元素添加至列表尾部
<code>list.insert(index, x)</code>	在列表指定位置index处添加元素x
<code>list.remove(x)</code>	在列表中删除首次出现的指定元素
<code>list.pop([index])</code>	删除并返回列表对象指定位置的元素
<code>list.clear()</code>	删除列表中所有元素，但保留列表对象，该方法在Python2中没有
<code>list.index(x)</code>	返回值为x的首个元素的下标
<code>list.count(x)</code>	返回指定元素x在列表中的出现次数
<code>list.reverse()</code>	对列表元素进行原地逆序
<code>list.sort()</code>	对列表元素进行原地排序
<code>list.copy()</code>	返回列表对象的浅拷贝，该方法在Python2中没有

```
list_var[index]  
x in list_var  
x not in  
list_var
```

列表推导式

- [表达式 for 变量 in 序列]

```
[ 2*k for k in range(100) ]
```

- [表达式 for 变量 in 序列 if 条件]

```
[ 2*k for k in range(100) if k % 10 == 0 ]
```

迭代器 (Iterator)

- 迭代器是一个可以记住遍历到位置的对象
- 从集合的第一个元素开始访问，直到所有的元素被访问完结束。迭代器只能往前不会后退。
- 迭代器有两个基本的方法
 - **iter()** 创建迭代器对象
 - **next()** 输出迭代器的下一个元素，没有更多的元素时抛出异常StopIteration

可迭代对象 (Iterable)

- 列表、元组、字典和集合都是可迭代的对象。
- 可以从中获取迭代器 (Iterator)

```
mystr = "banana"
myit = iter(mystr)
```

```
print(next(myit))
print(next(myit))
print(next(myit))
print(next(myit))
print(next(myit))
print(next(myit))
```

```
mystr = "banana"
for x in mystr:
    print(x)
```

```
>>> mystr = "banana"

>>> myit = iter(mystr)
>>> print(next(myit))
b
>>> print(next(myit))
a
>>> print(next(myit))
n
>>> print(next(myit))
a
>>> print(next(myit))
n
>>> print(next(myit))
a
>>> print(next(myit))
Traceback (most recent call last):
  File "<pyshell#9>", line 1, in <module>
    print(next(myit))
StopIteration
```

练习:迭代器

```
list=[1,2,3,4]
it = iter(list) # 创建迭代器对象
for x in it:
    print (x, end=" ")
```

```
import sys # 引入 sys 模块
list=[1,2,3,4]
it = iter(list) # 创建迭代器对象
while True:
    try:
        print (next(it))
    except StopIteration:
        sys.exit()
```

2.2 元组 (Tuple)

['tʌpəl]

列表的功能虽然很强大，但负担也很重，在很大程度上影响了运行效率。有时候我们并不需要那么多功能，很希望能有个轻量级的列表——元组 (tuple) 正是这样一种类型。

元组

card = ("♠", "8")

2.2 元组(Tuple)

- 元组和列表类似，但属于不可变序列。
- 元组一旦创建，用任何方法都不可以修改其元素（增加、删除、赋值）。
- 元组的定义方式和列表相同，但定义时所有元素是放在一对圆括号 “ (” 和 “) ” 中，而不是方括号中。
- 在不引起歧义时，括号可省略

```
>>>a_tuple = ('a', )
>>> a_tuple
('a')
>>>a_tuple = ('a', 'b', 'mpilgrim', 'z', 'example')
>>> a_tuple
('a', 'b', 'mpilgrim', 'z', 'example')
>>> x = ()          #空元组
>>> x
()
>>> x = 1,2,(3,4)
>>> x
(1, 2, (3, 4))
```

注意：

如果创建只有一个元素的元组，需要在元素后面加上一个逗号
“ , ” 。

```
>>> a=(3)    # 解释为表达式这和a = 3是一样的
```

```
>>> a
3
```

```
>>> a=3, #如果元组中只有一个元素，必须在后面多写一个逗号
```

```
>>> a
(3,)
```

2.2.1 元组创建与删除

- 使用tuple函数将其他序列转换为元组

```
>>> print(tuple('abcdefg'))
```

```
('a', 'b', 'c', 'd', 'e', 'f', 'g')
```

```
>>> aList = [-1, -4, 6, 7.5, -2.3, 9, -11]
```

```
>>> bTuple = tuple(aList)
```

```
(-1, -4, 6, 7.5, -2.3, 9, -11)
```

```
>>> s = tuple() #空元组
```

```
>>> s
```

```
()
```

- 使用del删除元组对象，不能删除元组元素

```
>>> aList = [-1, -4, 6, 7.5, -2.3, 9, -11]
```

```
>>> bTuple = tuple(aList)
```

```
>>> bTuple
```

```
(-1, -4, 6, 7.5, -2.3, 9, -11)
```

```
>>> del aList[0]
```

```
>>> aList
```

```
[-4, 6, 7.5, -2.3, 9, -11]
```

```
>>> del bTuple[0]
```

```
Traceback (most recent call last):
```

```
File "<pyshell#11>", line 1, in <module>
```

```
    del bTuple[0],
```

```
TypeError: 'tuple' object doesn't support item deletion
```

```
>>> del bTuple
```

```
>>>
```

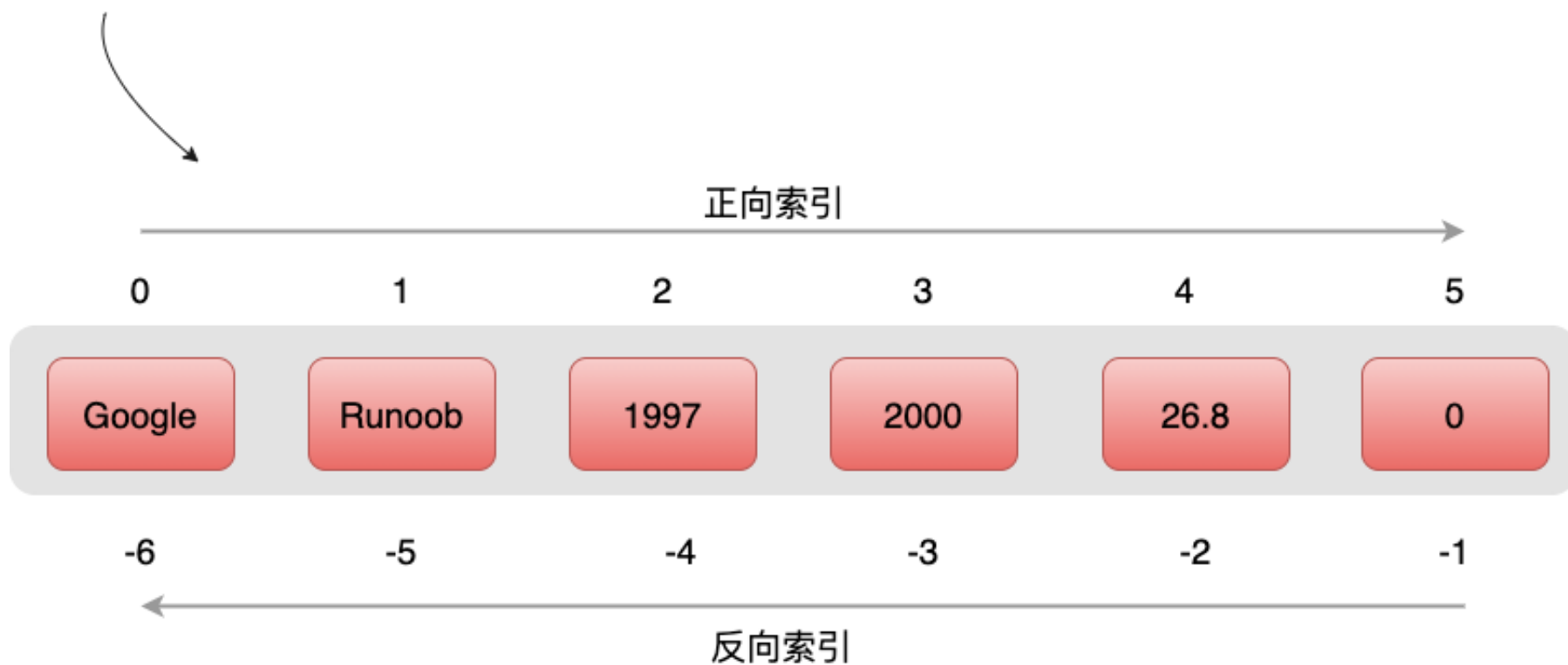
练习

- 元组创建与删除

访问元组元素

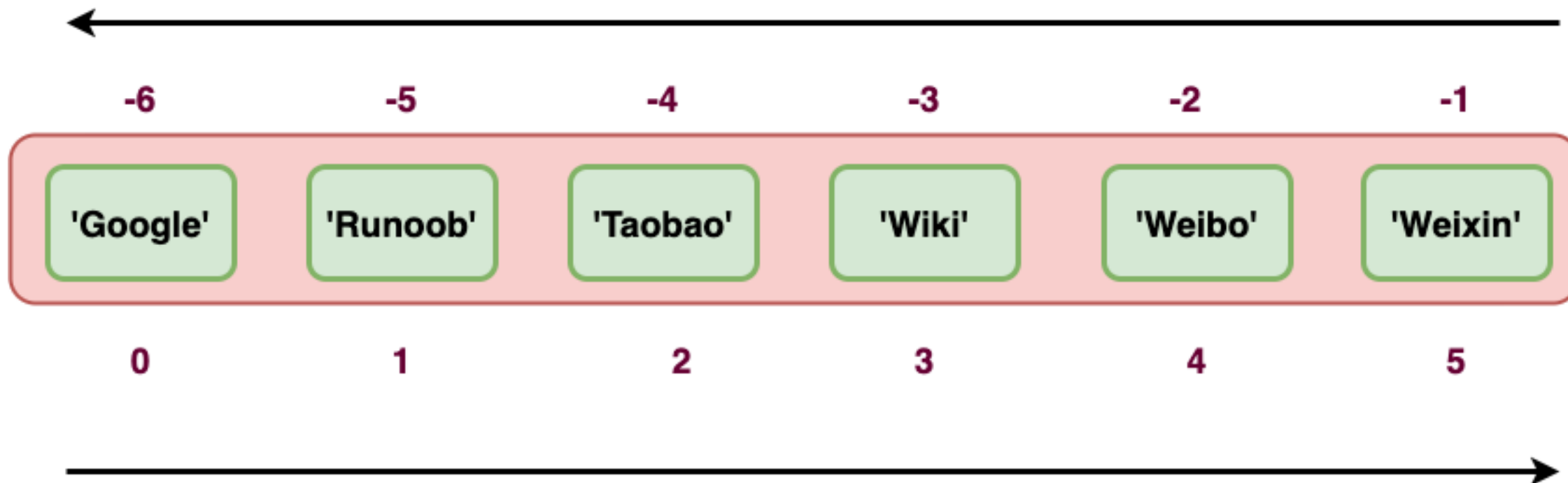
- 下标
 - t[1]

tup1 = ('Google', 'Runoob', 1997, 2000, 26.8, 0)



访问元组元素

- 切片
 - `t[2:6]`



切片

- 切片是Python序列的重要操作之一，适用于列表、元组、字符串、range对象等类型。

`slice[start:stop:step]`

元组访问

- 与list、str一样，可以访问某个元素，但是注意不能赋值

- 下标访问某个元素：

```
>>> x = tuple([x**2 for x in range(12)])  
(0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121)  
>>> x[5],x[-2]  
(25, 100)
```

- 切片访问多个元素

```
>>> x[-1::-1]  
(121, 100, 81, 64, 49, 36, 25, 16, 9, 4, 1, 0)  
>>> x[0:10:2]  
(0, 4, 16, 36, 64)
```

- tuple的元素可以是可变对象

- tuple不可变，指的是其元素不可变，但是元素指向的对象的可变并没有限制

```
>>> x = (1,2,[4,5])
```

```
>>> x[2] = [5,5]
```

TypeError: 'tuple' object does not support item assignment

```
>>> x[2][0] = 5
```

```
>>> x
```

```
(1, 2, [5, 5])
```

元组的不可变

- tuple不可变，指的是其元素不可变，但元素指向的对象的可变并没有限制
- tuple的元素可以是可变对象，这样tuple对象的值是可变的。

```
>>>x = ([1, 2], 3)
```

```
>>>x[0][0] = 5
```

```
>>>x
```

```
([5, 2], 3)
```

```
>>>x[0].append(8)
```

```
>>>x
```

```
([5, 2, 8], 3)
```

```
>>>x[0] = x[0] + [10,12]    #但是x[0] +=[10,12]可以，效用相当于x[0].extend([10,12])
```

TypeError: 'tuple' object does not support item assignment

报错！

错误说明：

列表的+运算会创建一个新的列表，对x[0]的赋值就是尝试修改元组元素的值（所指对象），所以这是不允许的，会报错。

练习

- 元组访问
- 元组的不可变

元组操作

- 除了索引（下标）和切片访问外，还支持其他序列的基本操作
- 连接和重复 + * (1,2) + ('a','b') (1,2) * 2
- 成员关系操作：
 - in, not in 1 in (1,2) '1' not in (1,2)
 - count(x): x出现的次数 (1,2).count(1)
 - index(value,[start,[stop]]) value在指定范围第一次出现的下标，找不到时抛异常ValueError。
(1,2).index(1)
- 比较运算： < <= == != >= >
- 内置函数：
 - sorted(iterable): 排序后返回新的list
 - reversed(iterable): 逆序后返回reversed迭代器
 - len(iterable)、max(iterable)、min(iterable)：长度，最大值，最小值
 - sum(iterable): 数值元素的和
 - enumerate(iterable): 返回枚举对象，每次返回包含下标和值的元组
 - zip(iter1,iter2...): 返回zip对象，每次返回各个可迭代对象中对应的相同位置的各个元素组成的元组

练习

- 元组操作

2.2.2 元组与列表的区别

- 元组中的数据一旦定义就**不允许更改**。(但可以重新进行赋值)
- 元组没有append()、extend()和insert()等方法，无法向元组中添加元素；
- 元组没有remove()或pop()方法，也无法对元组元素进行del操作，不能从元组中删除元素。
- 内建的tuple()函数接受一个列表参数，并返回一个包含同样元素的元组，而list()函数接受一个元组参数并返回一个列表。从效果上看，tuple()冻结列表，而list()融化元组。

2.2.2 元组的优点

- 元组的速度比列表更快。如果定义了一系列常量值，而所需做的仅是对它进行遍历，那么一般使用元组而不用列表。
- 元组对不需要改变的数据进行“写保护”将使得代码更加安全。
- 在后面介绍的字典(dict)中，元组可用作字典键（特别是包含字符串、数值和其它元组这样的不可变数据的元组）。而列表永远不能当做字典键使用，因为列表不是不可变的。

序列解包
(sequence unpacking)

序列解包(sequence unpacking)

- Python特有的语法，它将一个序列（或任何可迭代对象）解开，并将得到的值存储到一系列变量中。
- 一个函数或方法返回一个元组（或序列多其他可迭代对象），这时可以直接使用一组变量承接返回值。

序列解包(sequence unpacking)

$$\text{LHS} = \text{RHS}$$

- **LHS**，为对象引用，可以是变量名，可以是**通过下标或者切片描述的多个list元素**，通过**圆括号、方括号来组织**，通过**逗号来分割**
- **RHS**可以是任何可迭代对象，包括tuple、list、dict、range、str等

序列解包(sequence unpacking)

```
x, y, z = 1, 2, 3
```

```
(a, b), (c, d) = (1, 2), (3, 4)
```

```
a, b, c = 'abc'
```

```
a, b, c = [1, 2, 3]
```

```
a, b, c = (1, 2, 3)
```

```
a, b, c = {'a':1, 'b':2, 'c':3}
```

```
a, b, c = (x + 1 for x in range(3))
```

序列解包(sequence unpacking)

- 解包的序列包含的元素个数必须在等号左边列出的目标个数相同，否则Python将引发异常。
- 一个方法或函数返回序列时，其长度通常是不可预测的，可以使用通配符*来接收多余的值。(对象引用应该最多只出现一次，该引用前后的变量——对应赋值后，剩余的元素转变为list然后赋予该引用)

```
>>> a,b,c='abcd'
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    a,b,c='abcd'
ValueError: too many values to unpack (expected 3)
>>> |
```

```
a, b, *rest = (1, 2, 3, 4) # 使用*rest接收多余的
a, *rest, b = [1, 2, 3, 4] # *rest放在
a, *rest, b = [1, 4] # 序列没有足够的值
```

```
>>>rest
```

2.2.3 序列解包

- 序列解包中，变量引用可以是元素或者切片

```
>>> list1 = list(range(12)) # list1 = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
```

```
>>> x, y, list1[-1], list1[0:3] = 3,4,0,range(-5,0)
```

```
>>> list1
```

```
[-5, -4, -3, -2, -1, 3, 4, 5, 6, 7, 8, 9, 10, 0]
```

- 序列解包可以嵌套

```
>>> a,[b,(c,d)] = 1,['hello', ('Steve','Lee')]
```

```
>>> a,b,c,d
```

```
(1, 'hello', 'Steve', 'Lee')
```

- 序列解包对于字典同样有效：[后面再讲](#)

```
>>> b, c, d={'a':1,'b':2,'c':3}
```

```
>>> b, c, d
```

```
('a', 'b', 'c')
```

实验

- 序列解包

2.1.8 用于序列操作的常用内置函数

*review

- zip(**序列**1,**序列**2,...):将多个**序列**对应位置元素组合为元组，并返回包含这些元组的zip对象。Zip对象可以进一步用list()函数转换为列表对象。

```
>>> aList = [1, 2, 3]
```

```
>>> bTuple = (4, 5, 6,7)
```

```
>>> cZip = zip(aList, bTuple )
```

```
>>> cZip
```

```
<zip object at 0x0000000003728908>
```

```
>>> list(cZip)
```

```
[(1, 4), (2, 5), (3, 6)]
```

```
>>> list(cZip)  #迭代器已遍历结束，只能迭代一次.所以为空列表
```

```
[]
```

2.1.8 用于序列操作的常用内置函数

*review

- enumerate(**序列**)：枚举**序列**元素，返回枚举对象，其每个元素为包含下标和值的元组。该函数对元组、字符串同样有效。

```
>>>dList = [5, 6, 7]
>>> eEnum = enumerate(dList,1)
>>> eEnum
<enumerate object at 0x00000218EABB3B80>
>>> for item in eEnum:
    print(item)
(1, 5)
(2, 6)
(3, 7)
>>> list(eEnum) #迭代器已遍历结束，只能迭代一次.所以为空列表
[]
```

2.2.3 序列解包例子

```
>>> keys=['a','b','c','d']
>>> values=[1,2,3,4]
>>> for k,v in zip(keys,values):
    print(k,v)
```

```
a 1      #k,v=( 'a' ,1)
b 2      #k,v=( 'b' ,2)
c 3      #k,v=( 'c' ,3)
d 4      #k,v=( 'd' ,4)
```

```
>>> aList = [1,2,3]
>>> bList = [4,5,6]
>>> cList = [7,8,9]
>>> dList = zip(aList, bList, cList)
>>> for index, value in enumerate(dList):
    print(index, ': ', value)
```

```
0 : (1, 4, 7)      #index,value=(0,(1,4,7))
1 : (2, 5, 8)      #index,value=(1,(2,5,8))
2 : (3, 6, 9)      #index,value=(2,(3,6,9))
```

生成器表达式
(generator expression)

生成器表达式

- (表达式 for 变量 in 序列)
(2*k for k in range(100))

生成器表达式的结果是一个生成器对象。

```
>>> g = ((i+2)**2 for i in range(10)) #创建生成器对象
>>> g
<generator object <genexpr> at 0x0000000003095200>
```

生成器表达式

- 生成器对象类似于迭代器对象，具有惰性求值的特点，只在需要时生成新元素，比列表推导式具有更高的效率，空间占用非常少，尤其适合大数据处理的场合。

#生成器表达式更节省内存，适合操作大数据（如大文件读取）

```
(x for x in range(1000000))
```

#列表表达式会在内存一次性生成所有数据

```
[x for x in range(100)]
```

生成器表达式

- 使用生成器对象的元素时，可以根据需要将其转化为列表或元组
- 也可以使用生成器对象的next()方法或者内置函数__next__()进行遍历，或者直接使用for循环来遍历其中的元素。但是不管用哪种方法访问其元素，只能从前往后正向访问每个元素，不能再次访问

```
>>> tuple(g) #将生成器对象转换为元组
(4, 9, 16, 25, 36, 49, 64, 81, 100, 121)
>>> list(g) #生成器对象已遍历结束，没有元素了
[]
>>> g = ((i+2)**2 for i in range(10)) #重新创建生成器对象
>>> g.__next__() #使用生成器对象的__next__()方法获取元素
4
>>> g.__next__() #获取下一个元素
9
>>> next(g) #使用函数next()获取生成器对象中的元素
16
```

2.2.4 生成器表达式

- 使用for循环直接迭代生成器对象中的元素

```
>>> g = ((i+2)**2 for i in range(10))
>>> for item in g:                                     #使用循环直接遍历生成器对象中的元素
    print(item, end=' ')
4 9 16 25 36 49 64 81 100 121
```

- 访问过的元素不再存在

```
>>> g = ((i+2)**2 for i in range(10))
>>> 25 in g
True
>>> next(g)
36
>>> 25 in g      #访问过的元素不再存在
False
>>> next(g)
Traceback (most recent call last):
  File "<pyshell#27>", line 1, in <module>
    next(g)
StopIteration
```

生成器表达式：例子

- 毕达哥拉斯三元数组：存在 $\{x,y,z\}$, $0 < x < y < z$, 使得 $x^2 + y^2 = z^2$ 。求前10个毕达哥拉斯三元数组

```
pyt = [(x,y,z) for z in range(100) for y in range(1,z) for x in range(1,y) if x*x + y*y == z*z ]  
firstN_pyt = pyt[:10]  
print(firstN_pyt)
```

```
pyt = ((x,y,z) for z in range(1000) for y in range(1,z) for x in range(1,y) if x*x + y*y == z*z )  
firstN_pyt = [next(pyt) for x in range(10) ]  
print(firstN_pyt )
```

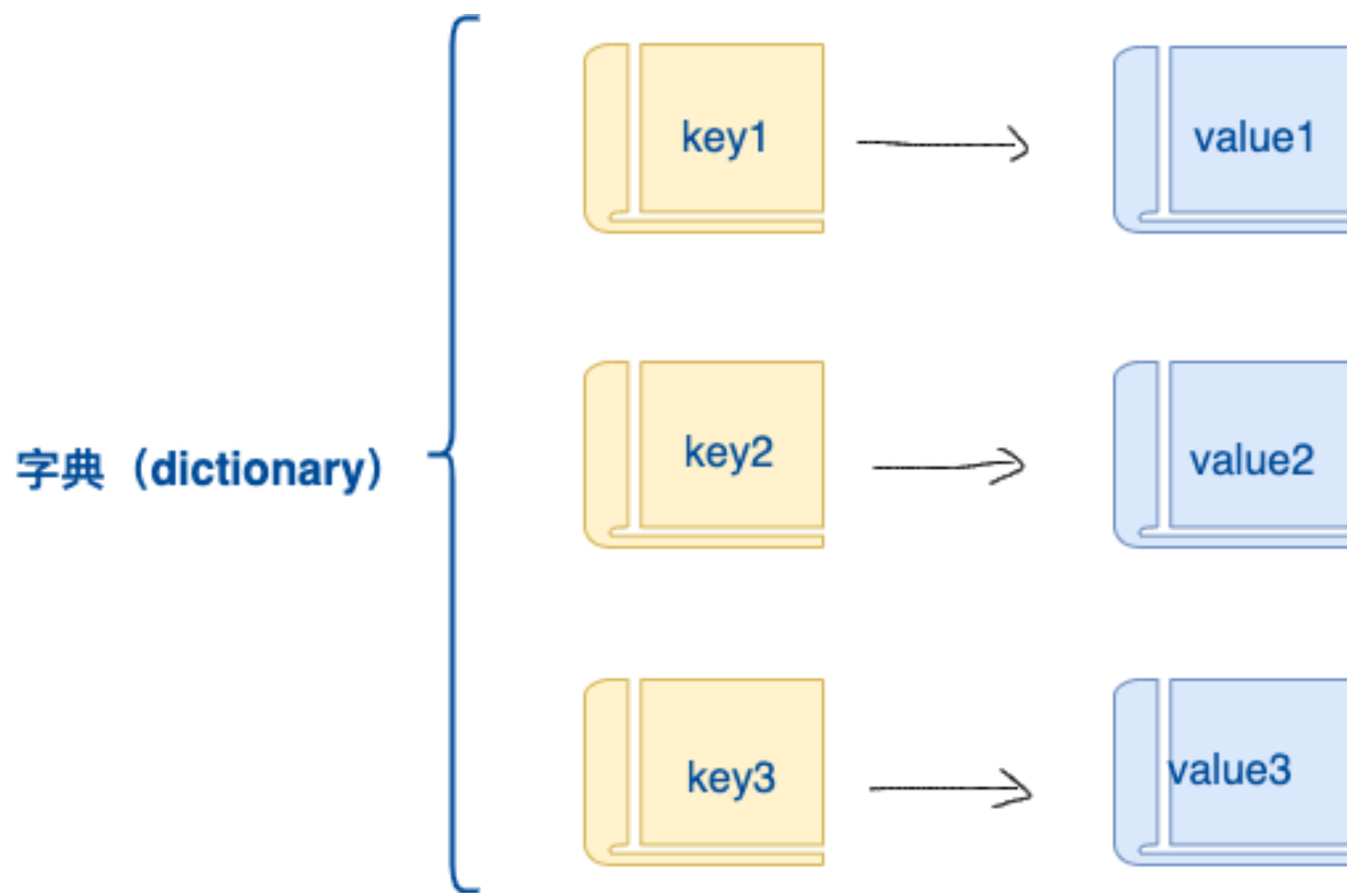
实验

- 生成器表达式

2.3 字典 (Dict)

字典

```
suits = {  
    "♠": "黑桃",  
    "♥": "红桃",  
    "♣": "梅花",  
    "♦": "方块"}  
}
```



字典

- 字典是包含键-值对的**无序的可变**序列，也称为映射（map）类型。给出了键和值的映射关系，表示一种映射或对应关系，也称关联数组。
- 字典中的每个元素包含两部分：**键和值**。向字典添加一个键的同时，必须为该键增添一个值。

2.3 字典(dict)

- **键(keys)不允许重复**
 - 可以为任意**不可变对象**（数据），比如整数、实数、复数、字符串、元组等。
 - 不要求不同元素的key为同一种类型
 - **无序**：不是按照key的大小顺序排列，实际上是按照key的hash值顺序
 - 可变对象**不能**作为key
- **值(values)**可以是任何对象，每个key对应着一个值，可以通过key查询到其对应的值

Python环境中许多内置函数和变量使用了dict
sys.modules 返回当前已经加载的模块名与模块对象的映射
globals()返回包含当前作用域内所有全局变量和值的字典
locals()返回包含当前作用域内所有局部变量和值的字典

2.3.1 字典的创建与删除

- 可以通过dict literal(字面值) 来创建dict对象：
 - 大括号界定，元素用key:value表示，元素间用逗号分隔
`{key1:value1 ,key2:value2, ... ,keyN:valueN }`

```
d1 = { 1:'food', 2:'drink', 3:'fruit' }  
d2 = {} #空字典  
d3 = {'name': 'Steve', 'age': 25, 'sex': 'male',  
      'address': {'city': 'shanghai', 'zip': '200433'},  
      1:'note1', 2:'note2',  
      '1': 'xx1', '2': 'xx2' }
```

2.3.1 字典的创建与删除

- 可以通过dict()函数来创建dict对象：
 - dict(): 创建空字典
 - dict(iterable): 新字典的元素来自于iterable, **每个元素**(可以为list、tuple、zip对象、enumerate等, 甚至也可以是长度为2的字符串)**必须包括两个子元素**
 - dict(mapping): 从一个字典对象mapping创建一个新的字典
 - dict(**kwargs): 关键字参数创建一个新的字典,
 - 关键字参数 keyvar=value对应着的key:value为 'keyvar':value

```
d4 = dict(['one',1],('two',2),['three',3])
z = zip(['one', 'two', 'three'], [1, 2, 3])
d5 = dict(z)
d6 = dict(['as','sd'])
d7 = dict({'three': 3, 'one': 1, 'two': 2})
d8 = dict(one=1, two=2, three=3)
```

2.3.1 字典创建与删除

- 通过dict类方法fromkeys创建新字典，每个元素的key来自于序列对象[序列中有重复的元素，将自动删除]，value均设置为value，缺省为None

`dict.fromkeys(seq[,value])`

```
>>> dict1=dict.fromkeys(['name','age','sex'])
```

```
>>>dict1
```

```
{'age': None, 'name': None, 'sex': None}
```

```
>>> dict2 = dict.fromkeys(range(5),10)
```

```
>>> dict2
```

```
{0: 10, 1: 10, 2: 10, 3: 10, 4: 10}
```

- **类方法**：通过**类名(dict)**来调用方法，无需首先创建对象的实例
- **实例方法**：通过实例对象来调用方法

- 使用del删除整个字典或者字典中的某个元素: `del dict2 ; del dict2[key]`
- `len(d)`: 返回字典中元素的个数

实验

- 字典的创建与删除

字典支持的操作

方法	说明
d[key]	返回key对应的value， key不存在raise KeyError
d.get(key[,default])	返回key对应的value， key不存在时返回default， 缺省为None
d.keys()	返回可迭代对象， 其中元素为字典中的所有key
d.values()	返回可迭代对象， 其中元素为字典中的所有value
d.items()	返回可迭代对象， 其中元素为包括了(key,value)的元组
key in d key not in d	判断字典d中有没有键==key， 返回True or False。 相当于 key in d.keys()
d[key] = value	设置字典中键=key的元素的值为value， 如果不存在， 则添加key:value
d.setdefault(key[,default])	如果key不在字典里插入新元素， 其值为default(缺省None)。如果在 不更新 。返回d[key]
d.update(other)	根据另一字典或元素为key/value对的可迭代对象更新字典， 返回None
del d[key]	删除元素， 如果key不存在， raise KeyError
d.popitem()	移走并返回某一个(key,value) 对
d.pop(key)	如果key存在删除对应元素并返回值， 否则raise KeyError
d.pop(key,value)	如果key存在删除对应元素并返回值， 否则返回value
clear()	清除所有元素
copy()	返回shallow copy后的新字典

2.3.2 字典元素的读取：下标

- 以键作为下标可以访问字典元素，若键不存在则抛出异常 `KeyError`

```
>>> aDict={'name':'Dong', 'sex':'male', 'age':37}
```

```
>>> aDict['name']
```

```
'Dong'
```

```
>>> aDict['tel']
```

```
Traceback (most recent call last):
```

```
File "<pyshell#53>", line 1, in <module>
```

```
    aDict['tel']
```

```
KeyError: 'tel'
```

2.3.2 字典元素的读取: get()

- `d.get(key[,default])` : 返回key对应的value, **key不存在**时返回default, 缺省为None

```
>>> aDict={'name':'Dong', 'sex':'male', 'age':37}
```

```
>>> print(aDict.get('address'))
```

None

```
>>> print(aDict.get('address',"have no address"))
```

have no address

```
>>> print(aDict.get('address', 'SDIBT'))
```

SDIBT

2.3.2 字典元素的读取

d.keys()	返回可迭代对象，其中元素为字典中的所有key
d.values()	返回可迭代对象，其中元素为字典中的所有value
d.items()	返回可迭代对象，其中元素为包括了(key,value)的元组
iter(d)	等价于iter(d.keys()), 即dict的迭代器是 针对所有key 的（不要求掌握，见备注）

```
d={'name':'Dong', 'sex':'male', 'age':37}
```

```
for item in d.items():  
    print(item)
```

```
('age', 37)  
( 'sex', 'male')  
( 'name', 'Dong')
```

```
for key,value in d.items():  
    print(key,value)
```

```
age 37  
sex male  
name Dong
```

```
for key in d.keys():  
    print(key)
```

```
age  
sex  
name
```

```
for value in d.values():  
    print(value)
```

```
37  
male  
Dong
```

```
>>> list(d.items())  
[('age', 37), ('sex', 'male'), ('name', 'Dong')]
```

对字典直接操作，字典的序列解包

-- 仅对key起作用

```
d={'name':'Dong', 'sex':'male', 'age':37}
```

```
for key in d:
    print(key)
```

age
sex
name

```
>>> list(d)
['age', 'sex', 'name']
```

- 序列解包对于字典同样有效：

```
>>> b, c, d={'a':1,'b':2,'c':3}
```

```
>>> b, c, d
```

```
( 'c ', 'a', 'b')
```

SHELL练习:dict内置方法,列表表达式

例如, 已知有一个包含一些同学成绩的字典, 计算成绩的最高分、最低分、平均分, 并查找所有最高分同学, 代码可以这样编写:

```
>>> scores = {"Zhang San": 45, "Li Si": 78, "Wang Wu": 40, "Zhou Liu": 96, "Zhao Qi": 65, "Sun Ba": 90, "Zheng Jiu": 78, "Wu Shi": 99, "Dong Shiyi": 60}
>>> highest = max(scores.values())
>>> lowest = min(scores.values())
>>> highest
>>> lowest
>>> average = sum(scores.values())*1.0/len(scores)
>>> average
>>> highestPerson = [name for name, score in scores.items() if score == highest]
>>> highestPerson
```

2.3.3 字典元素的添加、修改、删除：

d[key]=value

- 当以指定键为下标为字典赋值时，**若键存在**，则可以**修改**该键的值；若**不存在**，则表示**添加**一个键、值对。

```
>>> d[ 'age' ] = 38 #目前d为{'name':'Dong', 'sex':'male', 'age':37}
```

```
>>> d
```

```
{'age': 38, 'name': 'Dong', 'sex': 'male'}
```

```
>>> d['address'] = 'SDIBT'
```

```
>>> d
```

```
{'age': 38, 'address': 'SDIBT', 'name': 'Dong', 'sex': 'male' }
```

```
>>> aDict[ 'score' ] = aDict.get( 'score' ,[]) #和get方法的配合使用
```

```
>>> aDict['score'].append(98)
```

```
>>> aDict['score'].append(97)
```

```
>>> aDict
```

```
{'age': 37, 'score': [98, 97], 'name': 'Dong', 'sex': 'male'}
```

练习

- 下面的代码首先生成包含1000个随机字符的字符串，然后统计每个字符的出现次数。

```
import string
import random

x = string.ascii_letters + string.digits + string.punctuation
print('\n可使用的字符集:\n', x)

y = [random.choice(x) for i in range(1000)]
z = ''.join(y)

d = dict()

for ch in z:
    d[ch] = d.get(ch, 0) + 1  #千万不要使用d[ch] =d[ch] + 1，否则字典没有ch的key，出错

print('\n产生字符串为:\n',z,sep='')
print('\n统计结果为:\n',d,sep='')
```

2.3.3 字典元素的添加、修改、删除：

- d.update(another)使用字典对象的update方法将另一个字典another的键、值对 **修改** (key存在) 或 **添加** (key不存在) 到当前字典对象

```
>>> d= {'age': 37, 'score': [98, 97], 'name': 'Dong', 'sex': 'male'}
```

```
>>> d.update({'age':38,'city':'shanghai'})
```

```
>>> d
```

```
{'age': 38, 'score': [98, 97], 'name': 'Dong', 'city': 'shanghai', 'sex': 'male'}
```

- d.setdefault(key[,default]):如果key不在字典里插入新元素，其值为default(缺省None),并返回default；如果在**不更新**。返回d[key]

```
>>> d.setdefault('tel',"12345678")
```

```
'12345678'
```

```
>>> d
```

```
{'tel': '12345678', 'name': 'Dong', 'score': [98, 97], 'age': 37, 'sex': 'male'}
```

2.3.3 字典元素的添加、修改、删除

<code>del d[key]</code>	删除元素，如果key不存在，raise <code>KeyError</code>
<code>popitem()</code>	移走并返回某一个(key,value) 对
<code>pop(key)</code>	如果key存在删除对应元素并返回value，否则raise <code>KeyError</code>
<code>pop(key,value)</code>	如果key存在删除对应元素并返回值，否则返回value
<code>clear()</code>	清除所有元素

```
>>> d= {'tel': '12345678', 'name': 'Dong', 'score': [98, 97], 'age': 37, 'sex': 'male'}
```

```
>>> d.popitem()
```

```
('sex', 'male')
```

```
>>> d.pop('score',[])
```

```
[98, 97]
```

```
>>> d.pop('score',[])
```

```
[]
```

2.3.4 字典应用案例（可选，不讲）

- 也可以使用collections模块的defaultdict类（提供缺省值的dict）来实现该功能。

```
>>> import string
>>> import random
>>> x = string.ascii_letters + string.digits + string.punctuation
>>> y = [random.choice(x) for i in range(1000)]
>>> z = ''.join(y)
>>> from collections import defaultdict
>>> frequencies = defaultdict(int)
>>> frequencies
defaultdict(<type 'int'>, {})
>>> for item in z:
    frequencies[item] += 1
>>> frequencies.items()
```

2.3.4 字典应用案例（可选，不讲）

- 使用collections模块的Counter类可以快速实现这个功能，并且能够满足其他需要，例如查找出现次数最多的元素。下面的代码演示了Counter类的用法：

```
>>> from collections import Counter
```

```
>>> frequencies = Counter(z)
```

```
>>> frequencies.items()
```

```
>>> frequencies.most_common(1)
```

```
[('A', 22)]
```

```
>>> frequencies.most_common(3)
```

```
[('A', 22), (',', 18), (''', 17)]
```

2.3.5 有序字典（可选，不讲）

- Python内置字典是无序的，前面的示例很好地说明了这个问题。如果需要
一个可以记住元素插入顺序的字典，可以使用collections.OrderedDict。例
如下面的代码：

```
>>> x = dict() #无序字典
>>> x['a'] = 3
>>> x['b'] = 5
>>> x['c'] = 8
>>> x
{'b': 5, 'c': 8, 'a': 3}
```

```
>>> import collections
>>> x = collections.OrderedDict() #有序字典
>>> x['a'] = 3
>>> x['b'] = 5
>>> x['c'] = 8
>>> x
OrderedDict([('a', 3), ('b', 5), ('c', 8)])
```

sorted函数

- Sorted函数作用于字典只是将key排序后，返回key的列表

```
>>> d={'English':78, 'math':56, 'Chinese':37}
```

```
>>> sorted(d)
```

```
['Chinese', 'English', 'math']
```

- 如何按成绩排序？

```
>>> d={'English':78, 'math':56, 'Chinese':37}
```

```
>>> sorted(d.items())
```

```
[('Chinese', 37), ('English', 78), ('math', 56)]
```

```
>>> def key_func(item):  
    return item[1]
```

```
>>> sorted(d.items(),key=key_func)
```

```
[('Chinese', 37), ('math', 56), ('English', 78)]
```

字典推导式（解析式）

- 支持字典解析表达式来生成字典，如

- `{k:v for k,v in zip(['a','b','c'],[1,2,3])}`

```
>>> {k:v for k,v in zip(['a','b','c'],[1,2,3])}
{'a': 1, 'b': 2, 'c': 3}
```

- `{k:v for k,v in [('a',1),('b',2),('c',3)]}`

```
>>> {k:v for k,v in [('a',1),('b',2),('c',3)]}
{'a': 1, 'b': 2, 'c': 3}
```

实验

- 字典解析表达式：

`{c:c*4 for c in 'abcd' }` #生成一个新字典对象